

Static & Dynamic Analysis

Introduction

What are Static & Dynamic Analysis Methods?

Static analysis involves examining the code of a software application without executing it. The primary goal is to catch bugs and identify potential security vulnerabilities early in the development process. In contrast, dynamic analysis observes the application's behavior in real-time while it is running. Its aim is to uncover runtime issues such as performance bottlenecks, memory leaks, or unexpected behaviors that may not be apparent through code review alone. These methodologies are integral parts of the Mobile Application Security Testing Guide (MASTG).

	Static Analysis	Dynamic Analysis
Code Execution	No	Yes
When it's Used	Early in the SDLC	During or after deployment
What it Detects	Syntax errors, code smells, security vulnerabilities	Runtime bugs, performance issues
Manual Effort	Low	High (medium with automation)
Completeness of Coverage	Complete	Partial

Tools Needed for Analysis (Android)

- **Static Analysis Tools**

- Apktool is a critical tool used for reverse engineering Android APK files to extract resources and decode the manifest file.
- MASTG provides a structured approach for conducting both static and dynamic analysis.

- **Dynamic Analysis Tools**

- Android Studio and Android Emulator are used to set up controlled environments for executing and observing application behavior.
- Burp Suite is essential for intercepting and analyzing network traffic between the mobile application and backend APIs.

Why are these methods important?

- **Identification of Hidden Vulnerabilities**

- Static analysis can identify hardcoded credentials or sensitive data exposure in the source code before deployment.

- **Real-World Behavior Testing**

- Dynamic analysis allows testers to observe how the app handles real-world inputs and network conditions, uncovering runtime security flaws.

Case Study

Hardcoded API Key in a Retail Application

A popular retail application uses a third-party cloud service for managing inventory and user loyalty points. To simplify development, the developers integrate the service directly into the Android application.

The Vulnerability (Hardcoded Credentials)

During the development phase, the developers hardcode the administrative API key for the cloud service within the application's resource files. This action violates secure coding practices by leaving sensitive credentials accessible to anyone who can decompile the APK.

The Attack

A security researcher performs static analysis on the retail app. Using Apktool, they decompile the APK and search through the extracted resources. They locate the administrative API key stored in plain text within the `res/values/strings.xml` file. With this key, the researcher can theoretically gain full access to the backend inventory management system.

Result

The discovery demonstrates a critical vulnerability where sensitive credentials at rest are exposed. This case highlights the importance of using static analysis to identify hardcoded secrets that could lead to widespread backend compromise and data breaches.

Mitigations and Best Practices

- **Adherence to MASVS and MASTG**

- Follow the Mobile Application Security Verification Standard (MASVS) to define security requirements and use MASTG for systematic testing.

- **Automated and Manual Code Review**

- Integrate static analysis tools early in the Software Development Life Cycle (SDLC) to detect vulnerabilities like hardcoded secrets before they reach production.

- **Secure Storage of Credentials**

- Avoid hardcoding any sensitive information. Use platform-specific secure storage like Android Keystore for sensitive data and credentials.

Conclusion

Static and dynamic analysis are complementary methods essential for securing mobile applications. While static analysis helps in identifying structural and code-level vulnerabilities early, dynamic analysis provides insights into runtime behavior and network security. Combining these approaches within established frameworks like MASVS ensures a comprehensive security posture and protects applications against a wide range of mobile threats.

References

<https://www.qt.io/software-insights/static-vs.-dynamic-code-analysis>

<https://vfunction.com/blog/static-vs-dynamic-code-analysis/>